

DIAGNÓSTICO

ROL

Actua como desarrollador backend Node/Express.

CONTEXTO

Te voy a pasar una arquitectura en capas, pg sin ORM, ES modules.

TAREA

Necesito que identifiques la duplicacion y proponeme 2 o 3 estrategias de refactorización con sus pros y contras.

RESTRICCIONES

la API pública no debe cambiar", "los tests/Postman tienen que seguir pasando igual", "no rompas el patrón de LogHelper".

RESPUESTA 1

Estrategia 1: Herencia con Clases Base Genéricas

Consiste en crear una clase base para el Repositorio, otra para el Service, y una función constructora de Routers para el Controller, pasando el nombre de la tabla y la entidad como parámetros.

Cómo quedaría el código:

Para el repositorio, crearías un `base-repository.js`:

JavaScript

```
export default class BaseRepository {
  constructor(tableName, entityName) {
    this.table = tableName;
    this.entityName = entityName;
    this.db = new Db(); // Asumiendo tu Db de db-pg.js
  }

  getAllAsync = async () => {
    console.log(`${this.entityName}Repository.getAllAsync()`);
    return await this.db.queryAll(`SELECT * FROM ${this.table}`);
  }

  // ...getById, delete, etc.
}
```

⚖️ Pros y Contras

- o **Pros:** * Reducción drástica de líneas de código (un 70% menos de boilerplate).
 - o Mantiene los `console.log` dinámicos respetando el formato que venías usando.
 - o Si agregás una entidad nueva (ej: `profesores`), la creás en 5 minutos extendiendo la base.
- o **Contras:**
 - o Si las entidades empiezan a diferenciarse mucho en sus campos de la base de datos (por ejemplo, si `materias` tuviera columnas específicas que `cursos` no tiene), los métodos `create` y `update` genéricos se vuelven complejos de programar con SQL puro sin un ORM.

RESPUESTA 2

🔧 Estrategia 2: Middleware de Control de Errores y Funciones de Orden Superior (Para el Controller)

La mayor parte de la mugre del controlador son los bloques `try/catch` repetidos y el manejo de los `StatusCodes`. Podemos aislar el manejo de errores en un middleware global de Express y crear un generador de controladores.

Cómo quedaría el código:

Creamos un wrapper para evitar los `try/catch` (ej: `catchAsync.js`):

JavaScript

```
export const catchAsync = (fn) => (req, res, next) => {  
  fn(req, res, next).catch(next);  
};
```

Y centralizamos las respuestas en un controlador genérico que reciba el servicio:

JavaScript

```
// generador-controller.js  
export const createController = (service, entityName) => {  
  const router = Router();  
  
  router.get('/', catchAsync(async (req, res) => {  
    console.log(`${entityName}Controller.get`);  
    const returnArray = await service.getAllAsync();  
    returnArray ? res.status(StatusCodes.OK).json(returnArray)  
      : res.status(StatusCodes.INTERNAL_SERVER_ERROR).send("Error inte  
  }));  
  
  // ... los demás métodos (POST, PUT, DELETE con tu JSON de éxito)  
  return router;  
};
```

Pros y Contras

- **Pros:**
 - El ruteo y las validaciones de HTTP se vuelven 100% homogéneas.
 - Tu API pública no cambia en absoluto; Postman va a seguir respondiendo igual.
 - Eliminás el ruido visual de los `try/catch` duplicados en cada endpoint.
- **Contras:**
 - Si un endpoint requiere una validación de negocio muy particular antes de entrar al servicio, dependés de meter middlewares previos en el ruteador genérico, lo que puede volverse engorroso.

PROMPT

materias-repository

cursos-repository

alumnos-repository

ROL
Actua como desarrollador backend Node/Express.

CONTEXTO
Te voy a pasar una arquitectura en capas, pg sin ORM, ES modules. La estrategia para evitar la repeticion es esta: B) Repository por composición ("tiene un"), sin herencia
En vez de heredar, cada repository tiene adentro un objeto genérico que sabe hacer lo común, y al crearlo le pasa su nombre de tabla por parámetro. No hay clase madre/hija: hay un objeto que se parametriza y al que se le delega.

```
// Un repository genérico que sabe hacer lo común para CUALQUIER tabla
class GenericRepository {
  constructor(tabla) {
    this.tabla = tabla;
    this.db = new Db();
  }
  getAllAsync = async () => await this.db.queryAll(`SELECT * FROM ${this.tabla}`);
  // _getByIdAsync y deleteByIdAsync también viven acá
}

// CursosRepository "TIENE UN" GenericRepository
configurado con 'cursos':
class CursosRepository {
  constructor() {
    this.base = new GenericRepository('cursos'); // ← crea el objeto y le pasa la tabla
  }
  // Lo común se lo PIDE a su objeto interno (delega):
  getAllAsync = async () => await this.base.getAllAsync();

  // Y agrega lo suyo:
  createAsync = async (entity) => { /* INSERT INTO cursos (nombre) ... */ }
}

AlumnosRepository hace lo mismo: this.base = new GenericRepository('alumnos') y delega lo común.
```

 Diferencia con A: en A heredás ("es un"): un curso-repo es un repository base). En B tenés un objeto adentro y lo parametrizás ("tiene un"): un curso-repo tiene un repository genérico configurado con 'cursos'. Las dos eliminan la duplicación. Para este proyecto, A suele ser la más corta; B evita la herencia a cambio de delegar cada método común.

TAREA
Necesito que crees una clase GenericRepository que reciba el nombre de la tabla en el constructor y maneje los métodos comunes (getAllAsync, getByIdAsync, deleteByIdAsync). Luego, refactorizá cursos-repository.js, alumnos-repository y materias-repository.js para que implementen el patrón de composición (instanciando GenericRepository adentro de su constructor y delegándole los métodos comunes).

RESTRICCIONES
la API pública no debe cambiar", "los tests/Postman tienen que seguir pasando igual", "no rompas el patrón de Logi-helper".seguí usando pg con SQL crudo, no introduces un ORM

+ Preguntá a Gemini

Flash 

RESPUESTA

1. El Repositorio Genérico (`generic-repository.js`)

Creamos este nuevo archivo para centralizar la lógica repetitiva.

JavaScript

```
import Db from './db-pg.js';

export default class GenericRepository {
  constructor(tabla, contextName) {
    this.tabla = tabla;
    this.contextName = contextName; // Para mantener el formato exacto de los logs
    this.db = new Db();
  }

  getAllAsync = async () => {
    console.log(`${this.contextName}.getAllAsync()`);
    const sql = `SELECT * FROM ${this.tabla}`;
    return await this.db.queryAll(sql);
  }

  getByIdAsync = async (id) => {
    console.log(`${this.contextName}.getByIdAsync(${id})`);
    const sql = `SELECT * FROM ${this.tabla} WHERE id=${id}`;
    return await this.db.queryOne(sql, [id]);
  }

  deleteByIdAsync = async (id) => {
    console.log(`${this.contextName}.deleteByIdAsync(${id})`);
    const sql = `DELETE FROM ${this.tabla} WHERE id=${id}`;
    return await this.db.queryRowCount(sql, [id]);
  }
}
```

2. Repositorios Refactorizados

materias-repository.js

JavaScript



```
import Db from './db-pg.js';
import GenericRepository from './generic-repository.js';

export default class MateriasRepository {
  constructor() {
    console.log('Estoy en: MateriasRepository.constructor()');
    this.db = new Db();
    this.base = new GenericRepository('materias', 'MateriasRepository');
  }

  // Métodos delegados a la composición
  getAllAsync = async () => await this.base.getAllAsync();
  getByIdAsync = async (id) => await this.base.getByIdAsync(id);
  deleteByIdAsync = async (id) => await this.base.deleteByIdAsync(id);

  // Métodos específicos
  createAsync = async (entity) => {
    console.log('MateriasRepository.createAsync(${JSON.stringify(entity)})');
    const sql = `INSERT INTO materias (nombre) VALUES ($1) RETURNING id`;
    const values = [entity?.nombre ?? ''];
    return await this.db.queryReturnId(sql, values);
  }

  updateAsync = async (entity) => {
    console.log('MateriasRepository.updateAsync(${JSON.stringify(entity)})');
    const sql = `UPDATE materias SET nombre = $2 WHERE id = $1`;
    const values = [entity.id, entity?.nombre ?? ''];
    return await this.db.queryRowCount(sql, values);
  }
}
```

corsos-repository.js

JavaScript



```
import Db from './db-pg.js';
import GenericRepository from './generic-repository.js';

export default class CursosRepository {
  constructor() {
    console.log('Estoy en: CursosRepository.constructor()');
    this.db = new Db();
    this.base = new GenericRepository('corsos', 'CursosRepository');
  }

  // Métodos delegados a la composición
  getAllAsync = async () => await this.base.getAllAsync();
  getByIdAsync = async (id) => await this.base.getByIdAsync(id);
  deleteByIdAsync = async (id) => await this.base.deleteByIdAsync(id);

  // Métodos específicos
  createAsync = async (entity) => {
    console.log(`CursosRepository.createAsync(${JSON.stringify(entity)})`);
    const sql = `INSERT INTO corsos (nombre) VALUES ($1) RETURNING id`;
    const values = [entity?.nombre ?? ''];
    return await this.db.queryReturnId(sql, values);
  }

  updateAsync = async (entity) => {
    console.log(`CursosRepository.updateAsync(${JSON.stringify(entity)})`);
    const sql = `UPDATE corsos SET nombre = $2 WHERE id = $1`;
    const values = [entity.id, entity?.nombre ?? ''];
    return await this.db.queryRowCount(sql, values);
  }
}
```

JavaScript



```
import Db from './db-pg.js';
import GenericRepository from './generic-repository.js';

export default class AlumnosRepository {
  constructor() {
    console.log('Estoy en: AlumnosRepository-new.constructor()');
    this.db = new Db();
    this.base = new GenericRepository('alumnos', 'AlumnosRepository-new');
  }

  // Métodos delegados a la composición
  getAllAsync = async () => await this.base.getAllAsync();
  getByIdAsync = async (id) => await this.base.getByIdAsync(id);
  deleteByIdAsync = async (id) => await this.base.deleteByIdAsync(id);

  // Métodos específicos
  createAsync = async (entity) => {
    console.log(`AlumnosRepository-new.createAsync(${JSON.stringify(entity)})`);
    const sql = ` INSERT INTO alumnos (
      nombre
      ,
      apellido
      ,
      id_curso
      ,
      fecha_nacimiento
      ,
      hace_deportes
    ) VALUES (
      $1,
      $2,
      $3,
      $4,
      $5
    ) RETURNING id`;

    const values = [
      entity?.nombre ?? '',
      entity?.apellido ?? '',
      entity?.id_curso ?? 0,
      entity?.fecha_nacimiento ?? null,
      entity?.hace_deportes ?? 0
    ];

    return await this.db.queryReturnId(sql, values);
  }

  updateAsync = async (entity) => {
    console.log(`AlumnosRepository-new.updateAsync(${JSON.stringify(entity)})`);
    let id = entity.id;

    const previousEntity = await this.getByIdAsync(id); // Sigue funcionando int
    if (previousEntity == null) return 0;

    const sql = `UPDATE alumnos SET
      nombre
      = $2,
      apellido
      = $2`;
```


Al aplicar el patrón de composición, delegamos la responsabilidad de las consultas repetitivas de selección y eliminación. Así impactó en el tamaño de tus archivos:

Repositorio	Líneas Originales	Líneas Actuales	Ahorro de Código
materias-repository.js	44	31	-30%
cursos-repository.js	43	30	-30%
alumnos-repository.js	77	64	-17%
Total en Repositorios	164	125	-24% global